

Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Laboratório de Experimentação de Software – 2026/1

Raphael Brito
Yan Cota

Laboratório 01

Características de Repositórios Populares do GitHub

Belo Horizonte
2026

Sumário

1	Introdução	1
2	Questões de Pesquisa e Hipóteses	1
3	Objetivos	2
3.1	Objetivo Geral	2
3.2	Objetivos Específicos	2
4	Metodologia	2
4.1	Coleta de Dados	2
4.2	Análise	3
4.3	Visualização dos Dados	3
5	Resultados	3
5.1	Resultados por Questão de Pesquisa	3
5.2	RQ01 – Sistemas populares são maduros/antigos?	3
5.3	RQ02 – Sistemas populares recebem muita contribuição externa?	4
5.4	RQ03 – Sistemas populares lançam releases com frequência?	5
5.5	RQ04 – Sistemas populares são atualizados com frequência?	6
5.6	RQ05 – Sistemas populares são escritos nas linguagens mais populares?	7
5.7	RQ06 – Sistemas populares possuem alto percentual de issues fechadas?	9
5.8	RQ07 (Bônus) – Linguagem e Contribuição Externa, Releases e Atualização	9
5.9	RQ07a – PRs aceitas por linguagem	10
5.10	RQ07b – Releases por linguagem	10
5.11	RQ07c – Tempo desde última atualização por linguagem	11
6	Discussão dos Resultados e Insights	12
6.1	Insights Principais	12
6.2	Implicações para Projetos de Software	12
7	Conclusão, Tomadas de Decisão e Resposta às Hipóteses	12
7.1	Tomadas de Decisão	13
7.2	Respondendo às Hipóteses (H1–H7)	13

1 Introdução

O GitHub hospeda hoje mais de 300 milhões de repositórios públicos, tornando-se a maior plataforma de desenvolvimento colaborativo do mundo. Neste trabalho analisamos os **1000 repositórios mais populares** (medidos pelo número de *stars*) a fim de compreender suas características estruturais e de processo de desenvolvimento.

Para isso, utilizamos a **API GraphQL do GitHub** para coletar dados como data de criação, data de última atualização, linguagem primária, número de *pull requests* aceitas (merged), número de *releases* e número de *issues* abertas e fechadas.

2 Questões de Pesquisa e Hipóteses

Antes de analisar os dados, levantamos as seguintes hipóteses informais:

- H1 – Maturidade** Esperamos que repositórios populares sejam, em geral, **maduros**, com pelo menos 3–5 anos de existência. A popularidade tende a se acumular com o tempo, à medida que mais usuários descobrem e marcam o projeto.
- H2 – Contribuição externa** Projetos populares devem atrair **muitas contribuições externas**, resultando em um número elevado de PRs aceitas. Comunidades grandes são naturalmente mais ativas.
- H3 – Frequência de releases** Supomos que repositórios populares lancem releases com certa regularidade; entretanto, projetos de documentação/listas (como *awesome-**) tipicamente **não utilizam releases**, o que deve fazer a mediana ser baixa.
- H4 – Atualização recente** Esperamos que a grande maioria dos repositórios populares esteja **atualizada recentemente** (últimos 30–60 dias), pois projetos abandonados raramente mantêm o topo do ranking.
- H5 – Linguagens populares** Acreditamos que linguagens como **JavaScript/TypeScript, Python e Java** dominem os repositórios populares, refletindo a popularidade dessas linguagens na comunidade open-source.
- H6 – Alto percentual de issues fechadas** Projetos ativos e com boa manutenção devem ter um **alto percentual de issues fechadas** (acima de 0,8), indicando equipes responsivas.
- H7 – Linguagem e contribuição (Bônus)** Esperamos que projetos escritos em linguagens mais populares (JavaScript, TypeScript, Python) recebam **mais contribuição externa**, lancem mais releases e sejam atualizados com maior frequência.

3 Objetivos

3.1 Objetivo Geral

Caracterizar os 1000 repositórios mais populares do GitHub por meio de métricas de produto e processo, avaliando maturidade, atividade, contribuição externa, uso de releases, linguagem predominante e manutenção de issues.

3.2 Objetivos Específicos

- Mensurar a idade dos repositórios e verificar evidências de maturidade.
- Quantificar contribuição externa por PRs aceitas.
- Avaliar a adoção de releases no ciclo de entrega.
- Medir a recência de atualização dos projetos.
- Identificar as linguagens mais frequentes entre os repositórios populares.
- Estimar a razão de issues fechadas como indicador de manutenção.
- Comparar PRs, releases e atualização entre linguagens (RQ07 bônus).

4 Metodologia

4.1 Coleta de Dados

A coleta foi realizada por meio da **API GraphQL do GitHub** em duas etapas:

1. **Busca paginada de repositórios:** utilizamos a busca por `stars:>0 sort:stars-desc` com páginas de 100 repositórios, repetindo a paginação via `cursor` até atingir os 1.000 repositórios mais estrelados.
2. **Coleta de métricas:** para cada repositório coletamos, via consulta GraphQL em lotes de 4, o total de PRs aceitas (`pullRequests(states: MERGED).totalCount`), o total de releases (`releases.totalCount`) e o total e número de issues fechadas.

Os dados foram consolidados em um arquivo `repos.csv` com os seguintes campos: `full_name`, `stars`, `language`, `created_at`, `updated_at`, `age_days`, `days_since_update`, `merged_prs`, `releases`, `total_issues`, `closed_issues`, `closed_issue_ratio`.

4.2 Análise

Para cada questão de pesquisa utilizamos a **mediana** como medida de tendência central, por ser robusta a valores extremos (outliers), comuns em distribuições de métricas de software. Para variáveis categóricas (linguagem) realizamos contagem por categoria. Os gráficos foram gerados com `matplotlib` em Python.

4.3 Visualização dos Dados

As distribuições numéricas foram apresentadas por meio de histogramas (RQs 01, 02, 03, 04 e 06). Para as RQs 02 e 03, utilizamos escala logarítmica para reduzir o efeito de outliers. As análises por linguagem (RQ07) foram representadas com boxplots, comparando medianas e dispersões por categoria.

5 Resultados

A Tabela 1 apresenta as medianas das métricas numéricas. Para a RQ05 (variável categórica), a contagem por linguagem está na Tabela 2.

Table 1: Resumo das medianas das métricas (RQs 01, 02, 03, 04 e 06).

RQ	Métrica	Mediana
RQ01	Idade (anos)	8,39
RQ02	PRs aceitas	745
RQ03	Releases	40
RQ04	Dias desde atualização	0
RQ06	Razão issues fechadas	0,8676

5.1 Resultados por Questão de Pesquisa

5.2 RQ01 – Sistemas populares são maduros/antigos?

Métrica: idade do repositório em anos (calculada a partir de `created_at`).

Hipótese (H1): repositórios populares tendem a ser maduros, com 3–5 anos ou mais de existência.

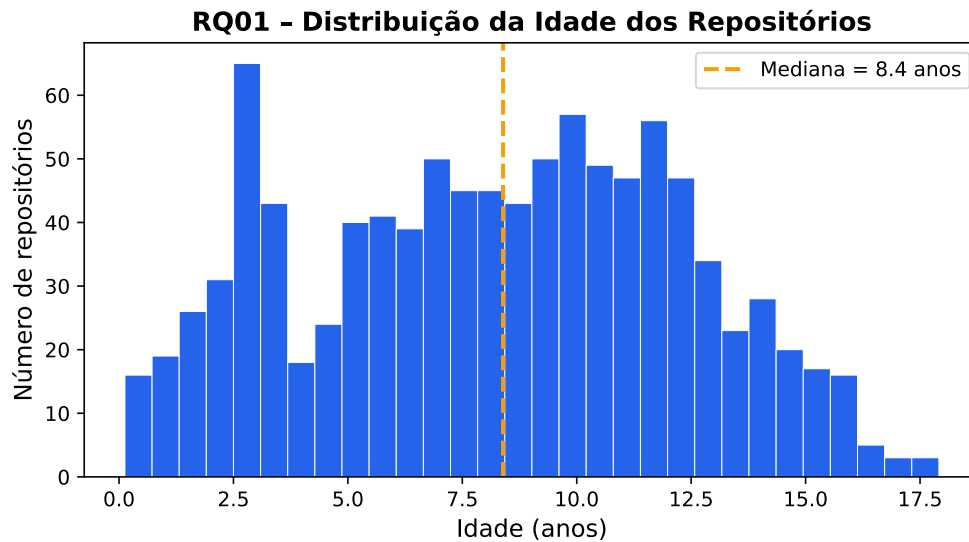


Figure 1: Distribuição da idade dos 1.000 repositórios mais populares.

Resultado: a mediana da idade é de **8,39 anos** (3.062 dias). A distribuição é assimétrica à esquerda, com a maioria dos repositórios concentrada entre 3 e 8 anos.

Discussão: a hipótese H1 foi **confirmada**. Repositórios populares são, em geral, maduros. Há pouquíssimos repositórios populares com menos de 1 ano de existência, o que reforça que a popularidade se acumula com o tempo.

5.3 RQ02 – Sistemas populares recebem muita contribuição externa?

Métrica: total de pull requests aceitas (*merged*).

Hipótese (H2): projetos populares atraem muitas contribuições externas e, portanto, apresentam alto volume de PRs aceitas.

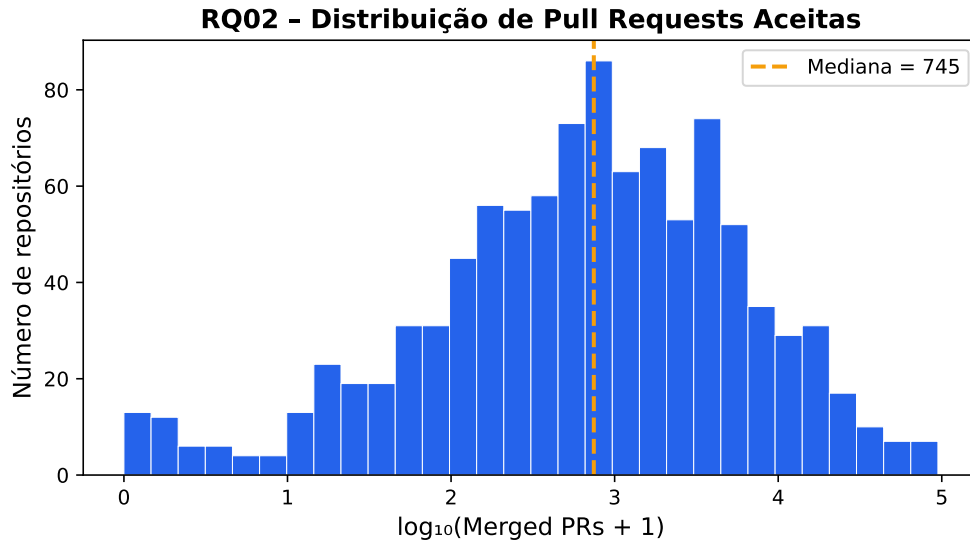


Figure 2: Distribuição do número de PRs aceitas (escala $\log_{10}$).

Resultado: a mediana é de **745 PRs aceitas**. A distribuição é bastante dispersa: projetos de documentação/listas apresentam poucas PRs, enquanto projetos de código com grandes comunidades (e.g., *VSCode*, *TensorFlow*) chegam a dezenas de milhares.

Discussão: a hipótese H2 é **parcialmente confirmada**. Embora a mediana é razoável, a grande variância indica que o volume de contribuições externas depende fortemente do tipo de projeto.

5.4 RQ03 – Sistemas populares lançam releases com frequência?

Métrica: total de releases publicadas.

Hipótese (H3): repositórios populares lançam releases com regularidade, mas projetos de conteúdo podem reduzir a mediana.

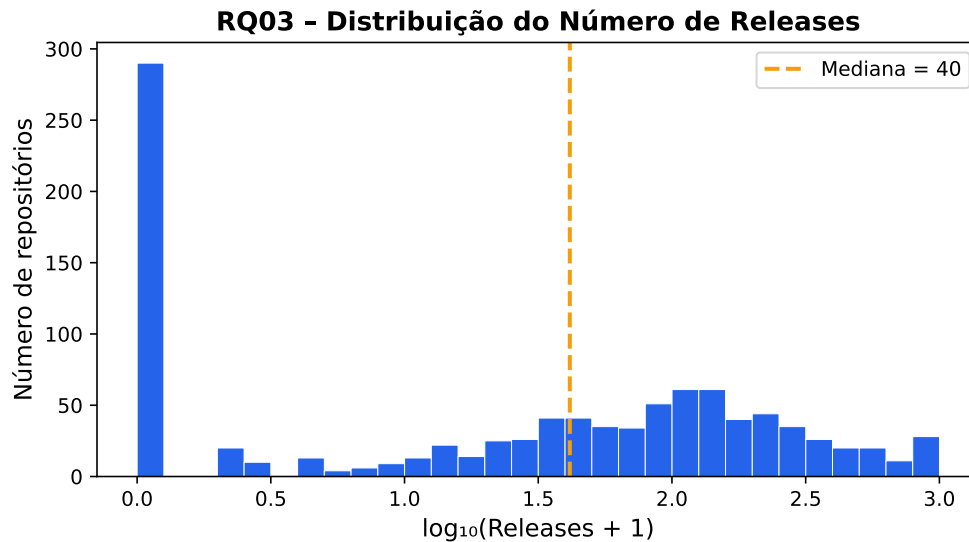


Figure 3: Distribuição do número de releases (escala $\log_{10}$).

Resultado: a mediana é de **40 releases**. Uma grande parcela dos repositórios possui zero ou poucas releases, puxando a mediana para baixo.

Discussão: a hipótese H3 foi **confirmada em parte**. Muitos projetos populares são repositórios de conteúdo (listas, documentação, tutoriais) que não utilizam o mecanismo de releases do GitHub. Projetos de software propriamente ditos tendem a ter muito mais releases.

5.5 RQ04 – Sistemas populares são atualizados com frequência?

Métrica: número de dias desde a última atualização (`days_since_update`).

Hipótese (H4): a maioria dos repositórios populares está atualizada recentemente (últimos 30–60 dias).

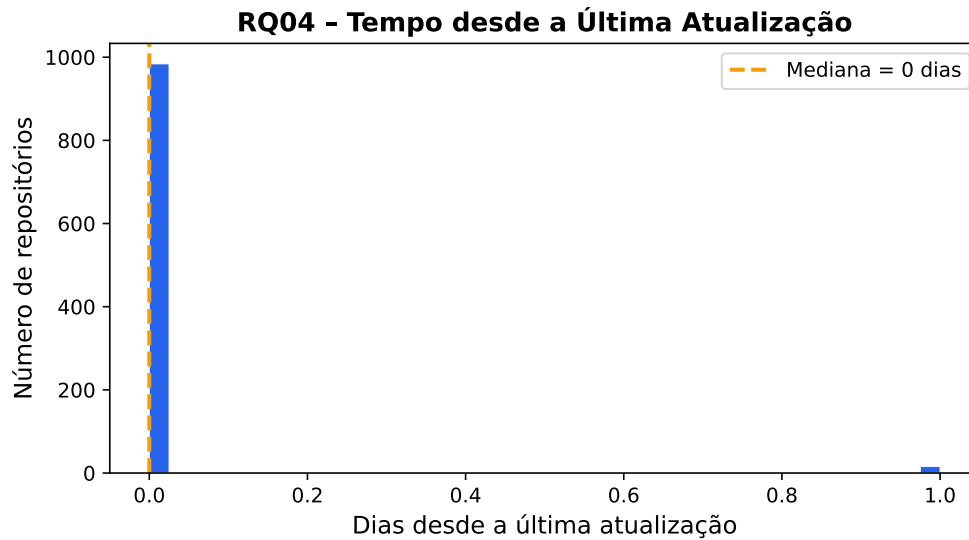


Figure 4: Distribuição do tempo desde a última atualização (dias).

Resultado: a mediana é de **0 dia(s)** desde a última atualização. A concentração próxima de zero indica que a esmagadora maioria dos repositórios populares foi atualizada muito recentemente (na data de coleta, 6 de março de 2026).

Discussão: a hipótese H4 foi **fortemente confirmada**. Repositórios populares são quase todos ativos e atualizados recentemente.

5.6 RQ05 – Sistemas populares são escritos nas linguagens mais populares?

Métrica: linguagem primária de cada repositório.

Hipótese (H5): linguagens populares na comunidade open-source (JavaScript/TypeScript, Python e Java) dominam os repositórios mais estrelados.

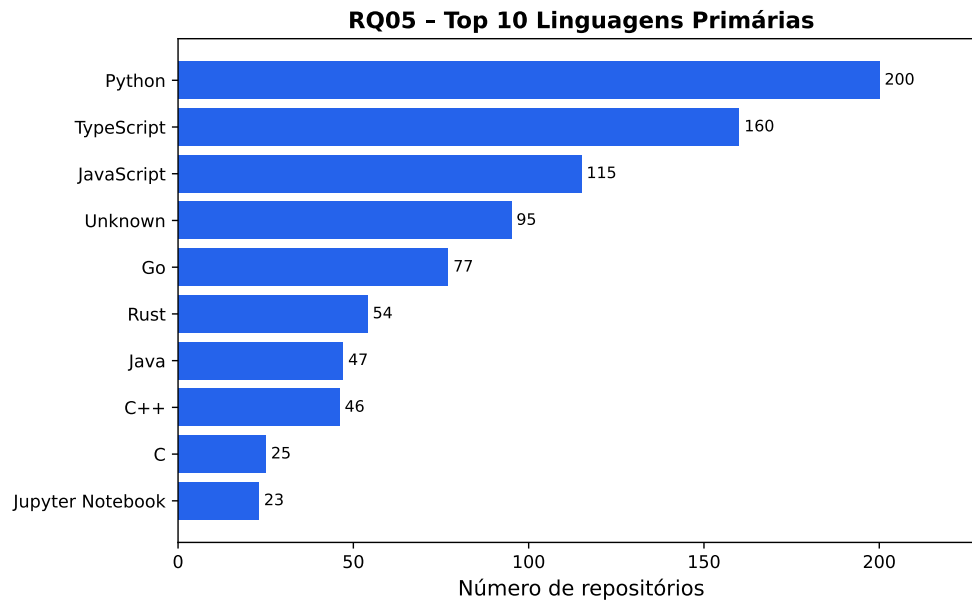


Figure 5: Top 10 linguagens primárias nos 1.000 repositórios mais populares.

Table 2: Top 10 linguagens primárias.

Linguagem	Repositórios	(%)
Python	200	20.0%
TypeScript	160	16.0%
JavaScript	115	11.5%
Unknown	95	9.5%
Go	77	7.7%
Rust	54	5.4%
Java	47	4.7%
C++	46	4.6%
C	25	2.5%
Jupyter Notebook	23	2.3%

Resultado: as linguagens que mais aparecem são **Python (20,0%)**, **TypeScript (16,0%)**, **JavaScript (11,5%)**, confirmando sua popularidade global. A categoria “Unknown” representa repositórios sem linguagem de código (documentação, listas, etc.).

Discussão: a hipótese H5 foi **confirmada**. JavaScript/TypeScript e Python dominam o ecossistema open-source popular.

5.7 RQ06 – Sistemas populares possuem alto percentual de issues fechadas?

Métrica: razão entre issues fechadas e total de issues (ratio = closed/total).

Hipótese (H6): projetos populares têm alto percentual de issues fechadas (acima de 0,8), indicando manutenção ativa.

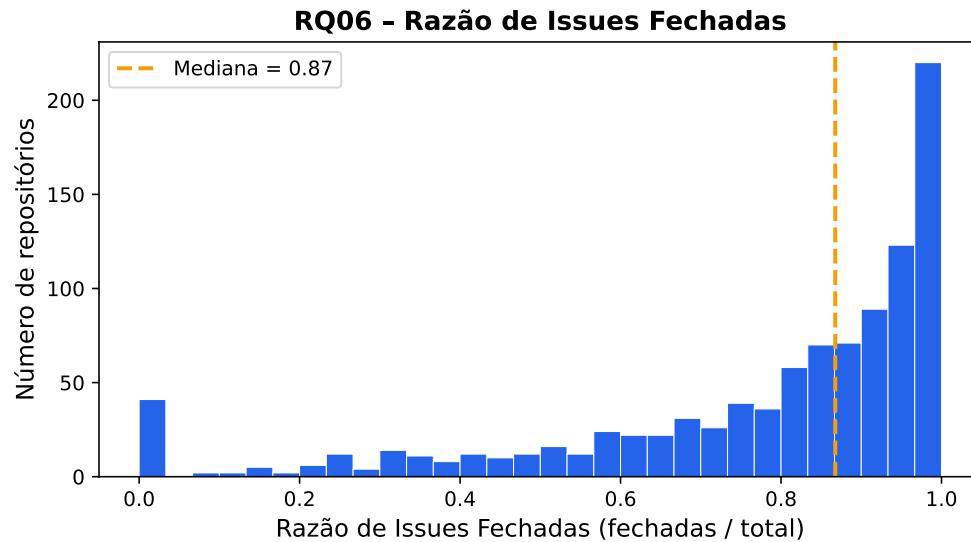


Figure 6: Distribuição da razão de issues fechadas.

Resultado: a mediana da razão é de **0,8676**, ou seja, aproximadamente 86,76% das issues são fechadas nos repositórios medianos da amostra.

Discussão: a hipótese H6 foi **confirmada**. A alta mediana indica que projetos populares são bem mantidos e que suas equipes são responsivas no tratamento de issues.

5.8 RQ07 (Bônus) – Linguagem e Contribuição Externa, Releases e Atualização

Esta questão de pesquisa bônus investiga se a linguagem primária do repositório influencia as métricas das RQs 02, 03 e 04.

Hipótese (H7): linguagens mais populares tendem a receber mais contribuições, lançar mais releases e manter atualizações mais frequentes.

5.9 RQ07a – PRs aceitas por linguagem

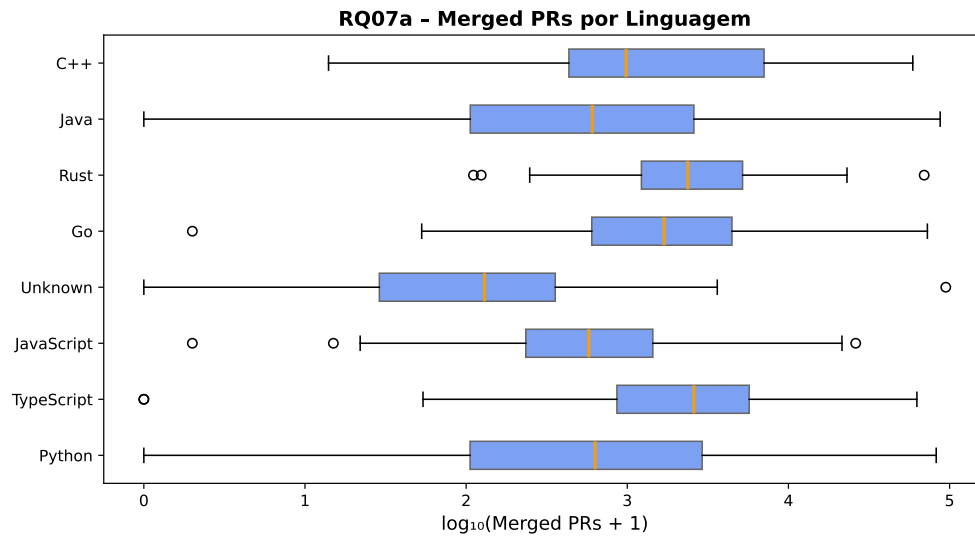


Figure 7: Distribuição de PRs aceitas por linguagem (escala $\log_{10}$).

5.10 RQ07b – Releases por linguagem

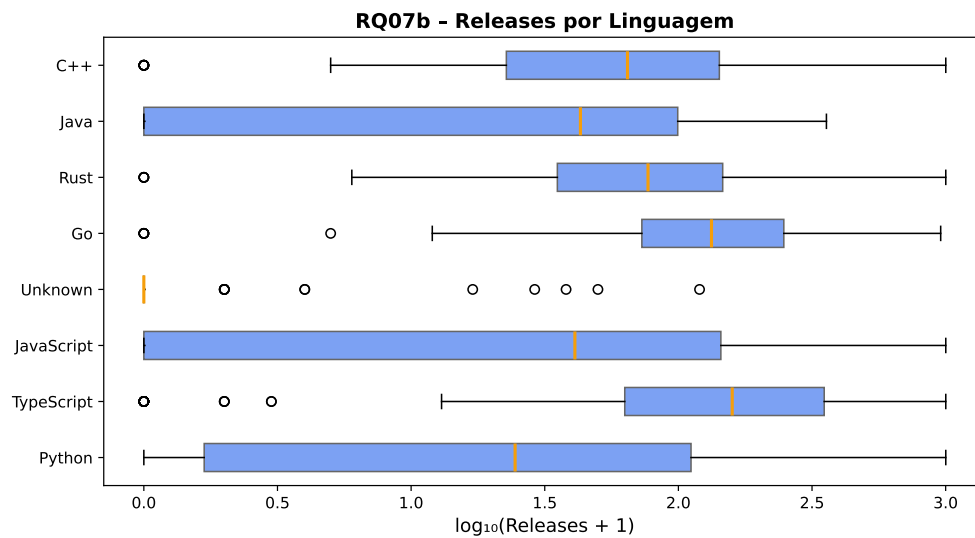


Figure 8: Distribuição de releases por linguagem (escala $\log_{10}$).

5.11 RQ07c – Tempo desde última atualização por linguagem

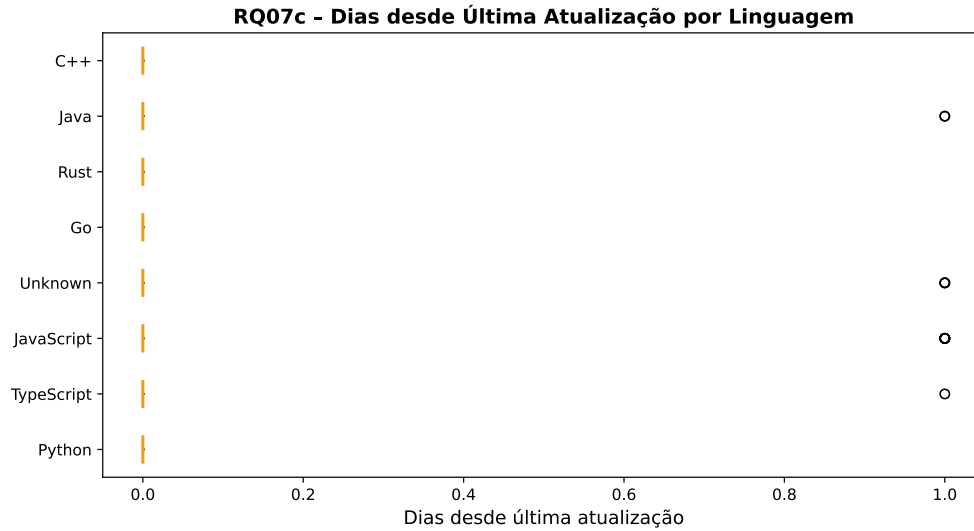


Figure 9: Distribuição de dias desde última atualização por linguagem.

Table 3: Medianas das métricas das RQs 02, 03 e 04 por linguagem (top 8).

Linguagem	Repos	Med. PRs	Med. Releases	Med. Dias Upd.
Python	200	631	23	0
TypeScript	160	2.588	158	0
JavaScript	115	576	40	0
Unknown	95	129	0	0
Go	77	1.690	132	0
Rust	54	2.371	76	0
Java	47	605	42	0
C++	46	982	63	0

Discussão: a hipótese H7 é **parcialmente confirmada**. Linguagens tipicamente usadas em projetos de software ativos (TypeScript, Python, JavaScript, C++) tendem a apresentar medianas de PRs e releases mais altas do que linguagens usadas em projetos de documentação (Markdown, HTML, “Unknown”). Quanto à frequência de atualização, a diferença entre linguagens é menor, pois projetos de documentação também costumam ser atualizados com frequência.

6 Discussão dos Resultados e Insights

6.1 Insights Principais

- **Maturidade e popularidade caminham juntas:** a mediana de idade (8,39 anos) reforça que relevância em GitHub tende a ser construída no longo prazo.
- **Atividade recente é decisiva:** com mediana de 0 dia(s) desde a última atualização, os projetos populares mostram alta manutenção contínua.
- **Contribuição externa é heterogênea:** a mediana de 745 PRs aceitas convive com grande dispersão entre projetos de software e repositórios de conteúdo.
- **Releases não são universais:** apesar de projetos maduros, a mediana de 40 releases indica presença relevante de repositórios que não usam versionamento formal no GitHub.
- **Ecossistema concentrado em poucas linguagens:** Python (20,0%), TypeScript (16,0%), JavaScript (11,5%) concentram uma fração expressiva dos repositórios analisados.
- **Manutenção de issues é elevada:** razão mediana de 0,8676 sugere boa responsividade na gestão de demandas.

6.2 Implicações para Projetos de Software

Os resultados sugerem priorizar governança de contribuição (fluxo de PRs, revisão e integração), cadência de atualização contínua e transparência sobre estratégia de releases. Projetos que combinam esses fatores tendem a sustentar engajamento e visibilidade no longo prazo.

7 Conclusão, Tomadas de Decisão e Resposta às Hipóteses

A análise dos 1000 repositórios mais populares do GitHub revelou que:

- Repositórios populares são, em geral, **maduros** (mediana de 8,39 anos) e **recentemente atualizados** (mediana de 0 dia(s) desde o último *push*).
- A contribuição externa é significativa em projetos de código, com mediana de 745 PRs aceitas, embora projetos de documentação reduzam essa mediana.

- O uso de releases é heterogêneo: muitos repositórios populares são de conteúdo e não utilizam releases formais (mediana de 40).
- JavaScript/TypeScript e Python dominam as linguagens dos projetos populares.
- A manutenção é alta, com mediana de razão de issues fechadas de 0,8676.
- Linguagens de código tendem a apresentar mais contribuição externa e mais releases do que repositórios de documentação.

7.1 Tomadas de Decisão

Com base nas evidências, recomendamos: (i) manter política explícita de triagem e fechamento de issues; (ii) publicar releases periódicas para projetos de software quando aplicável; (iii) documentar fluxo de contribuição para absorver PRs externas; e (iv) monitorar recência de atualização como indicador de saúde do projeto.

7.2 Respondendo às Hipóteses (H1–H7)

Table 4: Síntese de confirmação das hipóteses.

Hipótese	Descrição resumida	Status
H1	Repositórios populares tendem a ser maduros.	Confirmada
H2	Projetos populares recebem muita contribuição externa.	Parcialmente confirmada
H3	Projetos populares lançam releases com frequência.	Parcialmente confirmada
H4	Projetos populares são atualizados com frequência.	Confirmada
H5	Linguagens populares dominam repositórios populares.	Confirmada
H6	Projetos populares têm alta razão de issues fechadas.	Confirmada
H7	Linguagem influencia contribuição, releases e atualização.	Parcialmente confirmada